

LLM-Generated Lean4 Proofs: A Summer Research Retrospective

Abstract

This research paper presents a study of the application of Large Language Models (LLMs) in generating formal mathematical proofs using Lean4. Despite the growing capability of LLMs to generate code, producing verifiable Lean4 code remains a significant challenge due to its strict logical and type-theoretic requirements. The methodology involved a human-in-the-loop, iterative feedback process, where users provide oversight to help guide the LLM with error messages from Lean4 Infoview. Several LLMs were tested, including OpenAI's GPT-5, xAI's Grok 4, Google Gemini 2.5 Flash, Anthropic's Claude Sonnet 4, and a fine-tuned version of DeepSeek for proofs, DeepSeek Prover V2, on a series of mathematical theorems, documenting the progress and limitations. Key findings indicate that while LLMs can serve as a powerful tool in proof discovery, they often "hallucinate" deprecated or non-existent lemmas from the Mathlib, requiring significant human intervention. A comparison between the models reveals a trade-off between code-generation efficiency, cost-effectiveness, and the need for additional explanation. Ultimately, the research demonstrates that LLMs can lower the barrier to entry for formal verification by serving as a tool. Still, it is not fully capable of autonomous proof generation without human guidance.

1. Introduction

Absolute certainty has been a cornerstone of mathematics for centuries, relying on the construction of proofs to validate theorems. Traditionally, this has been a manual effort, one highly prone to error, difficult to scale, and human-centric. However, there are now interactive tools known as proof assistants that can be used to write and verify proofs through machine-assisted checking. Lean4 is a modern proof assistant that has garnered attention in recent years due to its expressive power and comprehensive community-contributed library called Mathlib. While Lean4's ability to provide absolute certainty is invaluable, it also comes with a steep learning curve. With complex syntax, tactics, and foundational concepts, there is a significant barrier to entry for many users, from students to professional mathematicians.

Recent work has primarily focused on autoformalization and proof automation. Projects like Herald and LeanDojo have concentrated on creating large datasets of Lean theorems and proofs to automate the translation of informal natural language into formal Lean4 statements. Other research, such as Lean Copilot and TheoremLlama, has explored using LLMs as "provers" by suggesting tactics or generating entire proofs in one single pass.

Although impressive, these projects are on complex, specialized frameworks of fine-tuned datasets. Most papers have evaluated success based on the LLM's ability to produce a correct proof independently, focusing on automation rates and success percentages. Additionally, several common challenges in recent work include LLM hallucinations, the scarcity of high-quality training data, and the difficulty of providing meaningful feedback to the model to improve generation.

Recent developments in Large Language Models (LLMs) have demonstrated remarkable skill in generating and debugging code across various programming languages. But can this be leveraged to overcome the entry barrier for formal verification? Through the usage of LLMs to generate Lean4 code, it might be possible to automate the proof-writing process, making formal verification accessible to a significantly larger audience. This research is motivated by the potential LLMs have in academic spaces, particularly in formal verification and proofs.

The main objective of this research project was to investigate the effectiveness of LLMs in writing correct and verifiable Lean4 code. I aimed to test several LLMs to compare their efficacy to gauge how applicable LLMs can be in the space. The primary focus is to consider what a general user might need. General users won't have access to large, complex datasets to fine-tune their LLM of choice; instead, they will have a prompt box. I will delve into how users can maximize the benefits of their LLM to support the Lean4 proof process.

This paper is structured in several sections. Section 2 discusses Lean4, LLMs, and the existing proof assistants currently in use. Section 3 describes the methodology throughout the research, including the different prompts I experimented with. Section 4 highlights the results of the experiment. (may need to add more here for the other sections, but I am not entirely sure)

2. Background & Related Work

Lean4 is an open-source, interactive theorem prover based on a dependent type theory utilizing the community-run Mathlib library. Unlike most programming languages, the purpose of Lean4 is to verify mathematical proofs and software correctness. Lean4 provides a system of tactics, which are commands that direct the proof-writing process into smaller subgoals. At the heart of Lean4 is a formal system that guarantees the validity of mathematical proofs; once verified by Lean4 InfoView, these proofs are mathematically sound.

The key component of Lean4 is its Mathlib library. Mathlib is an extensive, community-contributed collection of theorems, definitions, and proofs. Similar to a library for a programming language, the scale and quality of Mathlib make Lean4 a powerful tool for serious mathematical research. However, understanding and navigating Mathlib is a significant challenge for new users, requiring both a deep understanding of mathematical concepts, proof-writing, and its formal language.

ChatGPT, Gemini, Claude, and DeepSeek are all names that come to mind when we think of large language models (LLMs). Their use cases in programming have been growing steadily, as advancements in their programming capabilities continue to impress. This has been primarily demonstrated over the past year, with new "vibe-coding" projects emerging through platforms such as Cursor, Claude Code, OpenAI's Codex, and many others. However, LLMs like these can only be used to their fullest with the correct prompts and methods. For example, many engineers employ "few-shot prompting," a technique where the user provides context, including example code and results, to help the LLM generate a more desirable outcome. Context is essential for LLMs because it allows the model to understand what the user wants from the chat, whether it be more information to complete the desired task, strict directions for the LLM to follow, or something as simple as preferred code languages.

Large Language Models (LLMs), including prominent examples such as ChatGPT, Gemini, Claude, and DeepSeek, are becoming increasingly indispensable in the field of programming, driven by continuous advancements in their inherent capabilities. The preceding year has notably witnessed a surge in "vibe-coding" projects facilitated by platforms like Cursor, Claude Code, and OpenAI's Codex, thereby underscoring this trajectory of growth. Nevertheless, to fully leverage their potential, LLMs necessitate meticulously formulated prompts and methodologies. For example, "few-shot prompting" represents a technique in which users provide exemplary code and corresponding results as contextual information, which subsequently enables the LLM to generate more precise and desirable outputs. Context is paramount for LLMs, as it critically informs their comprehension of user intent, whether that involves task completion, adherence to stringent directives, or conformity to specified programming language conventions.

Several researchers in the past have fine-tuned their own models, whether a version of DeepSeek or something they created themselves, to achieve significantly better results on popular math competitions, such as the IMO and USAMO. However, these models are remarkably fine-tuned for writing mathematical proofs, not necessarily Lean4, following guidelines designed by the IMO and USAMO committees. Throughout my research, I came across various LLMs designed explicitly for this purpose, including Kimina Prover, DeepSeek Prover, and, more recently, ChatGPT's latest models.

3. Methodology

Throughout the time I spent researching LLMs usage to write Lean4, I utilized several models. I worked with Claude Sonnet 4, Chat-GPT 5, Gemini 2.5 Flash, xAI Grok 4, and most frequently DeepSeek Prover V2. Most of my prompting was done with the API platform OpenRouter with no specific configurations.

I went through several different prompting techniques. With ChatGPT-4o, I gave the LLM a role as a professional Lean4 programmer, providing specific guidelines it needed to follow: "You are a professional Lean4 programmer with a thorough knowledge of the Mathlib library and writing proofs in Lean4. I want you to construct a proof that proves that the limit as $1/x$ approaches infinity is zero.

You have a few guidelines:

1. Write the proof, but not in Lean4. Explain each step concisely and ensure correctness by citing sources.
2. Construct a plan to approach the Lean4 proof. Thoroughly review the Mathlib library for how you can put your formally written proof into the Lean4 language.
3. Write the proof in Lean4, including comments to explain the reasoning for future reference. Cite anything if possible.
4. Once Lean4 code is fully written, review the code and consider where errors can occur. Rather than retesting, we want the Lean4 proof to work on the first try. If you are not 100% confident in your proof, continue to review and refer to the Mathlib library if necessary. P.S. If there is any additional information you need, ask for it. I will not judge you for needing more data."

The goal for this prompt was that by specifying the role the LLM would play in the message chain, its results would generally improve, as it knows what I am specifically asking for before generating anything. With the guidelines, I considered what I would ask of a coworker if I wanted them to complete the same task I was giving to ChatGPT. By providing guidelines and

a step-by-step process, I could limit hallucinations. However, this did not work as I hoped.

For a fine-tuned LLM like DeepSeek Prover V2, would longer prompts be beneficial? I didn't think so. LLMs like DeepSeek Prover V2 are designed to generate math proofs without extra guidelines or specifications. So when I first started playing around with the LLM, I just asked: "Write a proof in Lean 4 of the fact that the limit of $(1/x) = 0$ as x tends to infinity." This had some impressive results, which, for a fine-tuned LLM, I wasn't surprised by. To note, I had prompted the LLM without WebSearch enabled.

The third prompt I tried was heavily inspired by a paper released on using LLMs to answer the Math Olympiad questions:

"Prove that the limit of $\sin x$, as $x \rightarrow \infty$, does not exist.

Core Instructions

**** Rigor is Paramount :** Your primary goal is to produce a complete and rigorously justified solution . Every step in your solution must be logically sound and clearly explained . A correct final answer derived from flawed or incomplete reasoning is considered a failure .

**** Honesty About Completeness :** If you cannot find a complete solution , you must **** not **** guess or create a solution that appears correct but contains hidden flaws or justification gaps . Instead , you should present only significant partial results that you can rigorously prove . A partial result is considered significant if it represents a substantial advancement toward a full solution . Examples include : * Proving a key lemma . * Fully resolving one or more cases within a logically sound case - based proof . 5 * Establishing a critical property of the mathematical objects in the problem . * For an optimization problem , proving an upper or lower bound without proving that this bound is achievable .

**** Use TeX for All Mathematics :** All mathematical variables , expressions , and relations must be enclosed in TeX delimiters (e . g . , ' Let n be an integer . ') .

Output Format

Your response **MUST** be structured into the following sections , in this exact order .

****1. Summary **** Provide a concise overview of your findings . This section must contain two parts :

**** a . Verdict :** State clearly whether you have found a complete solution or a partial solution.

**** For a complete solution :** State the final answer , e . g . , " I have successfully solved the problem . The final answer is ..."

**** For a partial solution :** State the main rigorous conclusion (s) you were able to prove , e . g . , " I have not found a complete solution , but I have rigorously proven that ..."

**** b . Method Sketch :** Present a high - level , conceptual outline of your solution . This sketch should allow an expert to understand the logical flow of your argument without reading the full detail . It should include : * A narrative of your overall strategy . * The full and precise mathematical statements of any key lemmas or major intermediate results . * If applicable , describe any key constructions or case splits that form the backbone of your argument .

****2. Detailed Solution **** Present the full , step - by - step mathematical proof . Each step must be logically justified and clearly explained . The level of detail should be sufficient for an expert to verify the correctness of your reasoning without needing to fill in any gaps . This section must contain ONLY the complete , rigorous proof , free of any internal commentary , alternative approaches , or failed attempts .

Self-Correction Instruction ### Before finalizing your output , carefully review your " Method Sketch " and " Detailed Solution " to ensure they are clean , rigorous , and strictly adhere to all instructions provided above . Verify that every statement contributes directly to the final , coherent mathematical argument .

Then, write a Lean4 proof using all of your findings.”

This was followed by a debugging prompt I had drafted. The goal was that if the LLM were as concise as possible, it would have fewer potential points of error.

“I want to try using more approaches using the Mathlib analysis library. Regarding your oscillation approach, `not_tendsto_of_tendsto_diff_const` is an unknown identifier.

Can you:

- Tell me if this is deprecated or replaced?
- Suggest the correct lemma or tactic from Mathlib.
- Update the code accordingly.”

Fourth, I figured that since LLMs often struggle with hallucinating, why not have them replace theorems that aren’t so sure on with “`apply?`”, a built-in feature to Lean4’s Infview that suggests several theorems from the Mathlib library. This was one of the prompts I used:

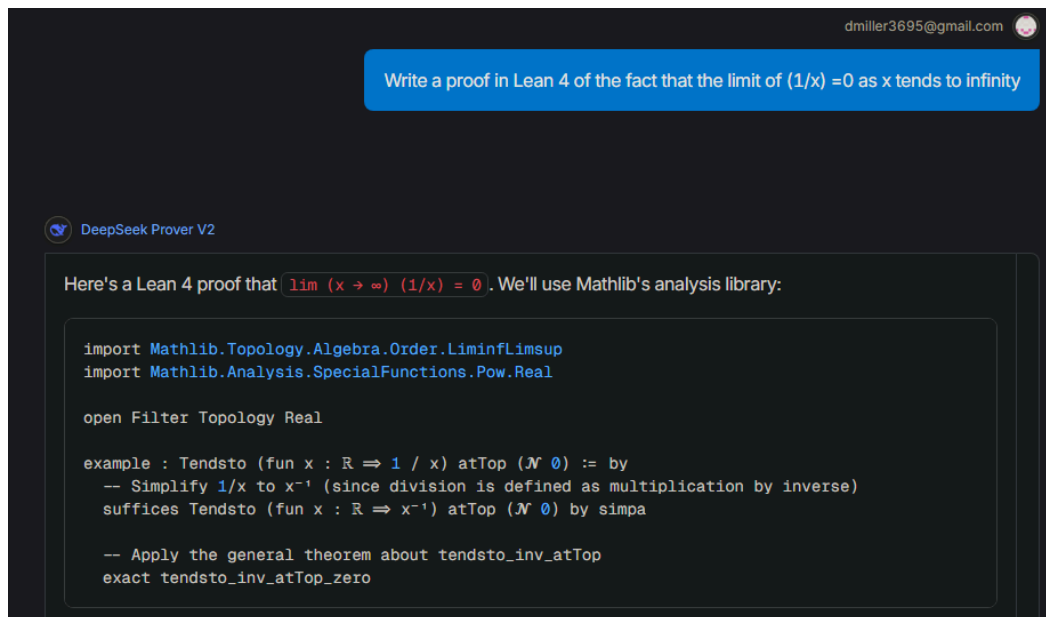
“Prove that if x tends to 2, then x^2 tends to 4 in Lean4, utilizing Lean4's “`apply?`” feature in place of theorems you are unsure about. I want you to prioritize these things:

Being concise to limit errors as much as possible

Using imports that actually exist, no Lean3 imports are used. Current, up-to-date imports”

So, what worked best? I found that specifying a role did not make a significant difference in the Lean4 code generation, so adding “You are a professional Lean4 programmer” is pointless. However, I have found guidelines to be generally helpful. It provides enough context for the LLM to generate what you want and how you want it. I was able to have DeepSeek

Prover V2 add comments to describe each step of the Lean4 proof, allowing me to understand its thought process more directly. However, something very drawn-out, like the Math Olympiad prompt, is generally too much to ask for a simple proof question. I suppose if it were a highly detailed proof, it could be helpful, but typically, we don't need the LLMs to share an outline of how they are going to solve the problem, as well as an additional detailed solution on top of that. It's essential to recognize that although these prompts may seem helpful, we are using more tokens (and therefore spending more money) with lengthy contextualization. The same can be said for enabling web search. This is ultimately a matter of preference, but I have found web searches to be best for models not fine-tuned on writing Mathematical proofs. LLMs such as Claude or ChatGPT 5 are great examples of LLMs that have demonstrated clearer code as a result. However, I can not say the same for DeepSeek Prover V2. In fact, DeepSeek Prover V2 was one of the only LLMs that provided a correct proof upon the first request.



dmiller3695@gmail.com

Write a proof in Lean 4 of the fact that the limit of $(1/x) = 0$ as x tends to infinity

DeepSeek Prover V2

Here's a Lean 4 proof that $\lim_{x \rightarrow \infty} (1/x) = 0$. We'll use Mathlib's analysis library:

```
import Mathlib.Topology.Algebra.Order.LiminfLimsup
import Mathlib.Analysis.SpecialFunctions.Pow.Real

open Filter Topology Real

example : Tendsto (fun x : ℝ => 1 / x) atTop (N 0) := by
  -- Simplify 1/x to x⁻¹ (since division is defined as multiplication by inverse)
  suffices Tendsto (fun x : ℝ => x⁻¹) atTop (N 0) by simpa

  -- Apply the general theorem about tendsto_inv_atTop
  exact tendsto_inv_atTop_zero
```

Key Points:

- Imports:** We use the relevant parts of Mathlib for topology and limits (`Filter`), real analysis, and exponentiation.
- Translation:**
 - `1/x` is expressed as `x⁻¹` in Lean (since division is defined via multiplication by the multiplicative inverse)
 - `atTop` represents the filter "as $x \rightarrow \infty$ "
 - `N 0` is the neighborhood filter around 0
- Proof Structure:**
 - We first rewrite the goal to use `⁻¹` notation
 - We then directly apply Mathlib's existing lemma `tendsto_inv_atTop_zero`
- Alternative Explicit Proof:**
If you wanted to write the ϵ -style proof manually (though in Lean this is less idiomatic than using built-in lemmas):

```
example : Tendsto (fun x : ℝ => 1 / x) atTop (N 0) := by
  rw [Metric.tendsto_nhds] -- Convert to ε-δ definition
  intro ε hε
  let M := 1 / ε + 1
  use M
  intro x hx
  simp
  have hM : 0 < M := by linarith [one_div_pos.mpr hε]
  calc |1 / x| = 1 / |x| := by norm_num
    _ ≤ 1 / M := by gcongr; exact le_of_lt hx; exact le_of_lt hM
    _ < ε := by field_simp [hε.ne.symm]; linarith
```

This second version manually handles the ϵ -M argument, showing Lean how to construct M for any given $\epsilon > 0$. The first version is preferred because it reuses Mathlib's optimized library lemmas.

Image Interactive App Landing Page 2D Game 3D Game

Start a new message...

Web search

The DeepSeek prover, without any additional context, highlighted key points, translation, and structure of the proofs it constructed, even providing an alternative proof using fewer built-in lemmas. The first proof was successful when verified using Lean4 InfoView. Although it cannot be said with certainty with the second proof, it is an excellent sign for LLM Lean4 usage.

```
1 import Mathlib.Topology.Algebra.Order.Liminf.Limsup
2 import Mathlib.Analysis.SpecialFunctions.Pow.Real
3
4 open Filter Topology Real
5
6 example : Tendsto (fun x : ℝ => 1 / x) atTop (N 0) := by
7   -- simplify 1/x to x⁻¹ (since division is defined as multiplication by inverse)
8   suffices Tendsto (fun x : ℝ => x⁻¹) atTop (N 0) by simp
9
10  -- Apply the general theorem about tendsto_inv_atTop
11  exact tendsto_inv_atTop_zero
```

Latest Mathlib

Mathlib-Demo-lean1130

Tactic state

No goals

Expected type

All Messages (0)

So what does this mean? Does prompt engineering matter for writing Lean4 code? Like I said, it depends. ChatGPT, Gemini, Claude, and other popular models can use extra context, considering they aren't models tuned to writing Lean4. DeepSeek Prover V2's fine-tuned nature makes extra-long prompts.]

Some of the mathematical theorems I tried to have the LLMs tackle included:
Proving that the limit of $1/x$ as $x \rightarrow \infty$ is 0

Prove that the square of a real number is always ≥ 0

Prove that the limit of $\sin x$ as $x \rightarrow \infty$ does not exist

Prove that if x tends to 2, then x^2 tends to 4

I evaluated the LLM's output by inputting the code into live/lean-lang.org and viewing the built-in Lean4 infoview to determine if the code had verified successfully or if errors were present. Most of the time, the code compiled, with some theorems or tactics recognized as “unknown identifiers.” Some of the proofs generated were efficient and concise, while others were fairly sloppy code.

4. Results

After testing GPT-5, Gemini 2.5 Flash, Claude Sonnet 4, DeepSeek Prover V2, and xAI Grok 4 on two different proofs of varying difficulty. I found that GPT-5 was the most efficient at solving the proof correctly. The first prompt I had the LLMs work on was proving that if x tends to 2, then x^2 tends to 4. One of the primary issues with all these LLMs was how the examples were declared. Here is the data I collected.

LLM Name	Number of Tries	Proof Status
Claude Sonnet 4	3	Successful
GPT-5	3	Successful
xAI Grok 4	4+	Unsuccessful
DeepSeek Prover V2	5+	Unsuccessful
DeepSeek Prover V2 (Web-Search)	5+	Unsuccessful
Gemini 2.5 Flash	5+	Unsuccessful

Claude Sonnet 4 and GPT-5 were the only successful LLMs of the 6 I tested. Claude Sonnet 4 began adding additional proofs (ultimately four) using different methods to demonstrate that if x tends to 2, then x^2 tends to 4. I did not prompt Claude to do this, which was interesting. GPT-5 worked just as well, with the second prompt mainly focused on fixing the declaration of the proof. Although it did not generate additional proofs without being asked, GPT-5's proof was straightforward and worked as instructed. This can't be said for the other LLMs. Although Claude Sonnet 4 and GPT-5 are the pricier options, Gemini 2.5 Flash and both versions of DeepSeek Prover V2 were unable to fix their proofs by the fifth try. This goes against my initial expectations because I assumed that since DeepSeek Prover V2 is a fine-tuned model for Lean4 and mathematical proofs, it wouldn't have a hard time generating a simple proof. Gemini 2.5 Flash is a commonly available LLM suitable for most general users, but it struggles with writing Lean4 code. xAI's Grok 4 was also not designed for writing Lean4 code and was the most expensive of all the LLMs. There are only 4+ tries because, as I continued to prompt Grok 4, the price of the prompting had increased significantly. For the results it generates, the price is not worth it for the end-user; one of the most common errors that all LLMs make is outdated imports in the code. For example, Claude Sonnet 4 would import Mathlib modules

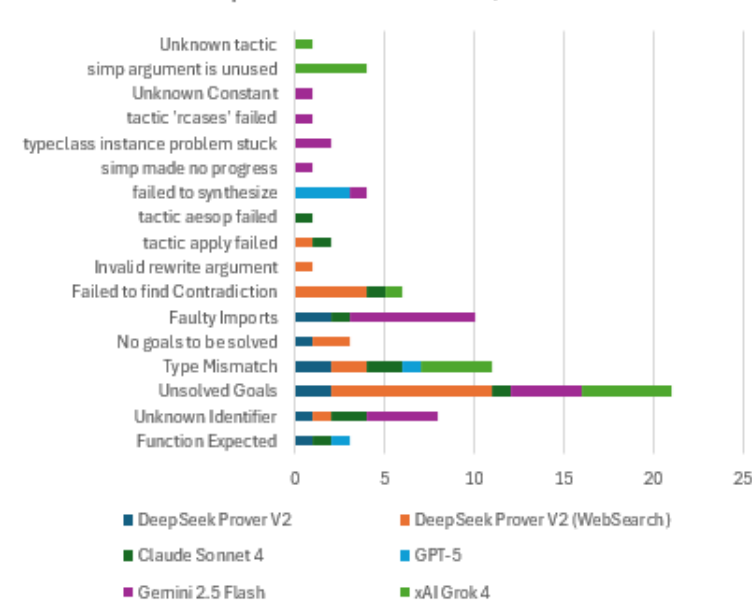
that no longer exist. To generally avoid this issue, I would use “import Mathlib”, which includes all currently existing modules. This issue arose several times, so by making this change, I avoided a standard error with all the LLMs. Additionally, the most common mistakes with all of the LLMs were type mismatches and unknown identifiers. Claude Sonnet 4, for example, attempted to use `continuous_at`, a tactic that doesn’t currently exist within Mathlib. There could be several explanations for this, one of which is that the data Anthropic trained Claude on is outdated, or Claude hallucinated the tactic.

Here is a table of all the errors that occurred from the different LLM’s

if x tends to 2, then x^2 tends to 4	DeepSeek Prover V2	DeepSeek Prover V2 (WebSearch)	Claude Sonnet 4	GPT-5	Gemini 2.5 Flash	xAI Grok 4	Totals:
Function Expected	1	0	1	1	0	0	3
Unknown Identifier	1	1	2	0	4	0	8
Unsolved Goals	2	9	1	0	4	5	21
Type Mismatch	2	2	2	1	0	4	11
No goals to be solved	1	2	0	0	0	0	3
Faulty Imports	2	0	1	0	7	0	10
Failed to find Contradiction	0	4	1	0	0	1	6
Invalid rewrite argument	0	1	0	0	0	0	1
tactic apply failed	0	1	1	0	0	0	2
tactic aesop failed	0	0	1	0	0	0	1
failed to synthesize	0	0	0	3	1	0	4
simp made no progress	0	0	0	0	1	0	1
typeclass instance problem stuck	0	0	0	0	2	0	2
tactic 'rcases' failed	0	0	0	0	1	0	1
Unknown Constant	0	0	0	0	1	0	1
simp argument is unused	0	0	0	0	0	4	4
Unknown tactic	0	0	0	0	0	1	1
Total:	9	20	10	5	21	15	

From this table, we can see that the most common errors among the LLMs were unsolved goals, type mismatch, and faulty imports. GPT-5 ultimately had the least amount of mistakes, with a successful proof, followed by DeepSeek Prover, which had nine errors but an unsuccessful proof.

Error Frequencies: If x tends to 2, x^2 tends to 4



The distributions of errors with the LLMs are highlighted in this chart.

Considering DeepSeek Prover and xAI were both unsuccessful and relatively inefficient, I decided to test the more easily accessible LLMs like GPT-5, Gemini 2.5 Flash, and Claude Sonnet 4 with a more difficult problem to prove: proving that the limit of $\sin x$, as $x \rightarrow \infty$, does not exist in Lean4. Proving this requires a more in-depth application of theorems and tactics than the previous prompt.

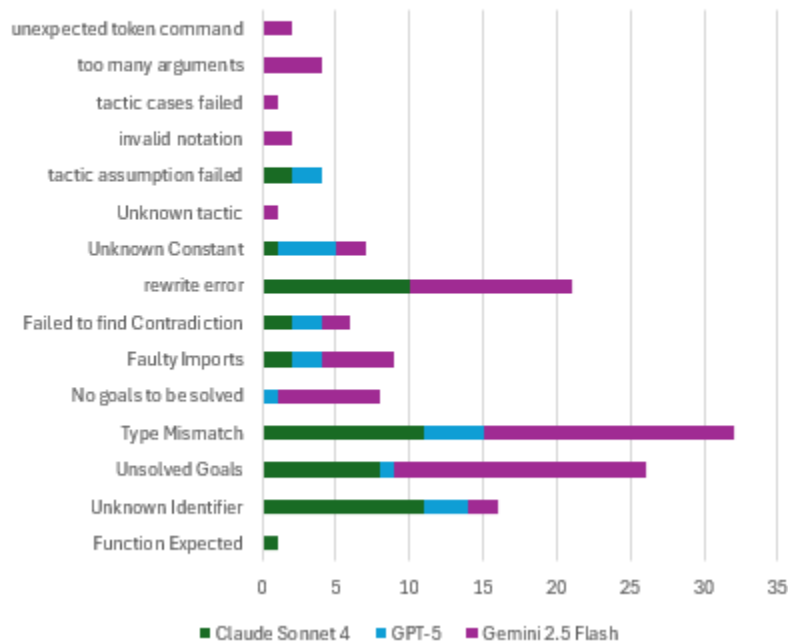
LLM Name	Number of Tries	Proof Status
Gemini 2.5 Flash	5+	Unsuccessful
GPT-5	3	Successful
Claude Sonnet 4	5+	Unsuccessful

Of the three remaining LLMs, only GPT-5 was able to write a correct proof in Lean4. Although it's the costliest of the three, GPT-5's ability to write both complex and straightforward mathematical proofs in Lean4 is a good sign for how LLMs are improving.

Here is a collection of data on the types of errors these LLMs produced.

lim of sin	Claude Sonnet 4	GPT-5	Gemini 2.5 Flash	Total:
Function Expected	1	0		1
Unknown Identifier	11	3	2	16
Unsolved Goals	8	1	17	26
Type Mismatch	11	4	17	32
No goals to be solved	0	1	7	8
Faulty Imports	2	2	5	9
Failed to find Contradiction	2	2	2	6
rewrite error	10	0	11	21
Unknown Constant	1	4	2	7
Unknown tactic	0	0	1	1
tactic assumption failed	2	2	0	4
invalid notation	0	0	2	2
tactic cases failed	0	0	1	1
too many arguments	0	0	4	4
unexpected token command	0	0	2	2
Total:	48	19	73	140

Error Frequencies: Proving that $\lim_{x \rightarrow \infty} \sin(x)$ as x approaches infinity does not exist.



Some key takeaways include that the most common errors in this type of proof were type mismatches, unsolved goals, and rewrite errors. Gemini 2.5 Flash also generated the most errors, partly due to its extremely long proofs. Overall, this demonstrates the capabilities of GPT-5 in writing Lean4 code and highlights its potential to aid individuals new to the Lean4 language in the coding process.

Here I am including the successful proofs generated by Claude Sonnet 4 and GPT-5

Proving that is x tends to 2, then x^2 tends to 4:

Claude Sonnet 4:

```

```lean
import Mathlib
open Filter Topology

-- First proof: Using continuity (corrected)
theorem limit_x_squared :
 Tendsto (fun x : ℝ => x^2) (ℵ 2) (ℵ 4) := by
 -- We need to show that (fun x => x^2) evaluated at 2 gives 4
 have h : (2 : ℝ)^2 = 4 := by norm_num
 rw [← h]
 -- Now use continuity of the squaring function
 exact (continuousAt_pow 2 2)

-- Alternative epsilon-delta proof (corrected)
theorem limit_x_squared_epsilon_delta :
 ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, |x - 2| < δ → |x^2 - 4| < ε := by
 intro ε he
 -- Choose δ = min 1 (ε / 5)
 use min 1 (ε / 5)
 constructor

```

```

· -- Prove $\delta > 0$
 apply lt_min
 · norm_num
 · exact div_pos hε (by norm_num : (0 : ℝ) < 5)

intro x hx
-- We have $|x - 2| < \min 1 (\varepsilon / 5)$
have h1 : $|x - 2| < 1$:= lt_of_lt_of_le hx (min_le_left _)
have h2 : $|x - 2| < \varepsilon / 5$:= lt_of_lt_of_le hx (min_le_right _)

-- Show that $|x + 2| < 5$
have bound_x_plus_2 : $|x + 2| < 5$:= by
 -- Use the triangle inequality: $|x + 2| = |(x - 2) + 4| \leq |x - 2| + |4|$
 calc |x + 2|
 = |(x - 2) + 4| := by ring_nf
 _ ≤ |x - 2| + |4| := abs_add _ _
 _ = |x - 2| + 4 := by norm_num
 _ < 1 + 4 := by linarith [h1]
 _ = 5 := by norm_num

-- Now prove $|x^2 - 4| < \varepsilon$
have factor : $x^2 - 4 = (x - 2) * (x + 2)$:= by ring
calc |x^2 - 4|
 = |(x - 2) * (x + 2)| := by rw [factor]
 _ = |x - 2| * |x + 2| := abs_mul _ _
 _ < ($\varepsilon / 5$) * 5 := mul_lt_mul' (le_of_lt h2) bound_x_plus_2 (abs_nonneg _) (div_pos hε (by
norm_num))
 _ = ε := by field_simp

-- Simple proof using continuity
```lean
example : Tendsto (fun x : ℝ => x^2) (ℕ 2) (ℕ 4) := by
  have h : (2 : ℝ)^2 = 4 := by norm_num
  rw [← h]
  exact (continuous_pow 2).continuousAt
```

```

Some interesting notes are that I had not requested multiple proofs from Claude Sonnet 4, but it generated four different proofs of what I had requested.

GPT-5:

Here are two corrected, compiling proofs.

1) Continuity pipeline (fixes the missing TopologicalSpace instance and pins the point before `.tendsto`)

```

```lean
import Mathlib.Topology.Instances.Real
import Mathlib.Topology.Algebra.Algebra
import Mathlib.Tactic

```

```

open Filter Topology

```

```

--  $\lim_{x \rightarrow 2} x^2 = 4$ 

```

```

example : Tendsto (fun x : ℝ => x ^ 2) (ℕ 2) (ℕ 4) := by
  -- continuity of squaring at 2
  have hAt : ContinuousAt (fun x : ℝ => x ^ 2) 2 :=
    (continuous_id.pow 2).continuousAt
  -- turn continuity-at into the limit to ℕ ((2)^2)
  have hlim : Tendsto (fun x : ℝ => x ^ 2) (ℕ 2) (ℕ ((2 : ℝ) ^ 2)) :=
    hAt.tendsto
  -- 2^2 = 4, so rewrite the target
  have hnum : ((2 : ℝ) ^ 2) = 4 := by norm_num
  simp [hnum] using hlim
  ...

```

Notes: This proof works once imports are fixed. I changed the imports just to import Mathlib.

The screenshot shows the Lean IDE interface. The editor on the left contains the following code:

```

1 import Mathlib
2
3 open Filter Topology
4
5 -- lim_{x → 2} x^2 = 4
6 example : Tendsto (fun x : ℝ => x ^ 2) (ℕ 2) (ℕ 4) := by
7   -- continuity of squaring at 2
8   have hAt : ContinuousAt (fun x : ℝ => x ^ 2) 2 :=
9     (continuous_id.pow 2).continuousAt
10  -- turn continuity-at into the limit to ℕ ((2)^2)
11  have hlim : Tendsto (fun x : ℝ => x ^ 2) (ℕ 2) (ℕ ((2 : ℝ) ^ 2)) :=
12    hAt.tendsto
13  -- 2^2 = 4, so rewrite the target
14  have hnum : ((2 : ℝ) ^ 2) = 4 := by norm_num
15  simp [hnum] using hlim

```

The right sidebar shows the 'Tactic state' with 'No goals' and 'Expected type'.

2) Tactic-flavored version (fixes the “type mismatch” by rewriting the endpoint)

```

lean
import Mathlib.Topology.Instances.Real
import Mathlib.Tactic

open Filter Topology

example : Tendsto (fun x : ℝ => x ^ 2) (ℕ (2 : ℝ)) (ℕ 4) := by
  -- get continuity via the `continuity` tactic
  have hCont : Continuous fun x : ℝ => x ^ 2 := by
    simp using (by continuity : Continuous fun x : ℝ => x ^ 2)
  have hAt : ContinuousAt (fun x : ℝ => x ^ 2) 2 := hCont.continuousAt
  -- first, the limit goes to ℕ ((2)^2)
  have hlim : Tendsto (fun x : ℝ => x ^ 2) (ℕ 2) (ℕ ((2 : ℝ) ^ 2)) :=
    hAt.tendsto
  -- rewrite ((2)^2) to 4 to match the goal
  have hnum : ((2 : ℝ) ^ 2) = 4 := by norm_num
  simp [hnum] using hlim
  ...

```

Notes: Second proof also works really well. I just had to fix imports, like the first one.

The screenshot shows the Lean IDE interface. The editor on the left contains the following code:

```

1 import Mathlib
2
3 open Filter Topology
4
5 -- lim_{x → 2} x^2 = 4
6 example : Tendsto (fun x : ℝ => x ^ 2) (ℕ 2) (ℕ 4) := by
7   -- continuity of squaring at 2
8   have hAt : ContinuousAt (fun x : ℝ => x ^ 2) 2 :=
9     (continuous_id.pow 2).continuousAt
10  -- turn continuity-at into the limit to ℕ ((2)^2)
11  have hlim : Tendsto (fun x : ℝ => x ^ 2) (ℕ 2) (ℕ ((2 : ℝ) ^ 2)) :=
12    hAt.tendsto
13  -- 2^2 = 4, so rewrite the target
14  have hnum : ((2 : ℝ) ^ 2) = 4 := by norm_num
15  simp [hnum] using hlim

```

The right sidebar shows the 'Tactic state' with 'No goals' and 'Expected type'.

As you can tell by my notes, GPT-5 wasn't as good as Claude Sonnet 4 with fixing the imports, but also generated more than one proof.

To prove that the limit of $\sin(x)$ as x approaches infinity does not exist, here we have GPT-5's proof:

```

```lean
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic
import Mathlib.Topology.Algebra.Order
import Mathlib.Data.Real.Archimedean
import Mathlib.Tactic

open Filter
open scoped Topology

/-- `sin x` does not have a (finite) limit as `x → ∞`. -/
theorem sin_no_limit_atTop : ¬ ∃ l : ℝ, Tendsto Real.sin atTop (𝓃 l) := by
 classical
 intro h
 rcases h with ⟨l, hl⟩

 -- Basic positivity used below
 have hpos : 0 < 2 * Real.pi := by
 have : 0 < (2 : ℝ) := by norm_num
 exact mul_pos this Real.pi_pos
 have hne : (2 * Real.pi) ≠ 0 := ne_of_gt hpos

 -- First subsequence: a n = π/2 + (n : ℝ) * (2π), so sin(a n) = 1
 let a : ℕ → ℝ := fun n => Real.pi / 2 + (n : ℝ) * (2 * Real.pi)

 have ha_mono : Monotone a := by
 intro m n hmn
 dsimp [a]
 have hmn' : (m : ℝ) ≤ n := by exact_mod_cast hmn
 have : (m : ℝ) * (2 * Real.pi) ≤ n * (2 * Real.pi) :=
 mul_le_mul_of_nonneg_right hmn' (le_of_lt hpos)
 linarith

 have ha_unbounded : ∀ M : ℝ, ∃ N : ℕ, M ≤ a N := by
 intro M
 -- Choose N ≥ (M - π/2)/(2π)
 obtain ⟨N, hN⟩ := exists_nat_ge ((M - Real.pi / 2) / (2 * Real.pi))
 refine ⟨N, ?_⟩
 -- Multiply by positive (2π) to get M - π/2 ≤ N·(2π)
 have hmul : M - Real.pi / 2 ≤ (N : ℝ) * (2 * Real.pi) := by
 have hN' : (M - Real.pi / 2) / (2 * Real.pi) ≤ (N : ℝ) := by
 exact_mod_cast hN
 have := mul_le_mul_of_nonneg_right hN' (le_of_lt hpos)
 -- ((M - π/2)/(2π)) * (2π) ≤ (N:ℝ) * (2π)
 simpa [div_eq_mul_inv, mul_comm, mul_left_comm, mul_assoc, hne] using this
 -- Add π/2 to both sides to get M ≤ a N
 have : M ≤ Real.pi / 2 + (N : ℝ) * (2 * Real.pi) := by
 simpa [sub_eq_add_neg, add_comm, add_left_comm, add_assoc] using
 add_le_add_left hmul (Real.pi / 2)
 simpa [a] using this

```

```

have ha_tendsto : Tendsto a atTop atTop :=
 tendsto_atTop_atTop_of_monotone ha_mono ha_unbounded

-- Compose: $\sin \circ a \rightarrow l$
have h1 : Tendsto (fun n => Real.sin (a n)) atTop ($\mathcal{N} l$) := h1.comp ha_tendsto

-- Use periodicity: $\sin(x + 2\pi \cdot k) = \sin x$
have hsin_a : $\forall n, \text{Real.sin } (a n) = 1$:= by
 intro n
 have : $\text{Real.sin } (a n) = \text{Real.sin } (\text{Real.pi} / 2)$:= by
 -- with $k = (n : \mathbb{Z})$
 simp [a] using Real.sin_add_int_mul_two_pi (Real.pi / 2) (n : $\mathbb{Z}$)
 simp [Real.sin_pi_div_two] using this

-- Uniqueness of limits gives $l = 1$
have hl_eq1 : $l = 1$:= by
 have hconst_to_l : Tendsto (fun _ : $\mathbb{N} \Rightarrow (1 : \mathbb{R})$) atTop ($\mathcal{N} l$) := by
 simp [hsin_a] using h1
 have hconst_to_1 : Tendsto (fun _ : $\mathbb{N} \Rightarrow (1 : \mathbb{R})$) atTop ($\mathcal{N} (1 : \mathbb{R})$) :=
 tendsto_const_nhds
 exact tendsto_nhds_unique hconst_to_l hconst_to_1

-- Second subsequence: $b n = \pi/2 + \pi + (n : \mathbb{R}) * (2\pi)$, so $\sin(b n) = -1$
let b : $\mathbb{N} \rightarrow \mathbb{R} := \text{fun } n \Rightarrow \text{Real.pi} / 2 + \text{Real.pi} + (n : \mathbb{R}) * (2 * \text{Real.pi})$

have hb_mono : Monotone b := by
 intro m n hmn
 dsimp [b]
 have hmn' : $(m : \mathbb{R}) \leq n$:= by exact_mod_cast hmn
 have : $(m : \mathbb{R}) * (2 * \text{Real.pi}) \leq n * (2 * \text{Real.pi})$:=
 mul_le_mul_of_nonneg_right hmn' (le_of_lt hpos)
 linarith

have hb_unbounded : $\forall M : \mathbb{R}, \exists N : \mathbb{N}, M \leq b N$:= by
 intro M
 obtain ⟨N, hN⟩ := exists_nat_ge ((M - (Real.pi / 2 + Real.pi)) / (2 * Real.pi))
 refine ⟨N, ?_⟩
 have hmul : $M - (\text{Real.pi} / 2 + \text{Real.pi}) \leq (N : \mathbb{R}) * (2 * \text{Real.pi})$:= by
 have hN' : $(M - (\text{Real.pi} / 2 + \text{Real.pi})) / (2 * \text{Real.pi}) \leq (N : \mathbb{R})$:= by
 exact_mod_cast hN
 have := mul_le_mul_of_nonneg_right hN' (le_of_lt hpos)
 simp [div_eq_mul_inv, mul_comm, mul_left_comm, mul_assoc, hne] using this
 have : $M \leq \text{Real.pi} / 2 + \text{Real.pi} + (N : \mathbb{R}) * (2 * \text{Real.pi})$:= by
 simp [sub_eq_add_neg, add_comm, add_left_comm, add_assoc] using
 add_le_add_left hmul (Real.pi / 2 + Real.pi)
 simp [b] using this

have hb_tendsto : Tendsto b atTop atTop :=
 tendsto_atTop_atTop_of_monotone hb_mono hb_unbounded

-- Compose: $\sin \circ b \rightarrow l$
have h2 : Tendsto (fun n => Real.sin (b n)) atTop ($\mathcal{N} l$) := h1.comp hb_tendsto

```

```

-- Periodicity and $\sin(x+\pi) = -\sin x$ give $\sin(b n) = -1$
have hsin_b : $\forall n, \text{Real.sin } (b n) = -1$:= by
 intro n
 -- First reduce by adding integer multiples of 2π
 have : $\text{Real.sin } (b n) = \text{Real.sin } (\text{Real.pi} / 2 + \text{Real.pi})$:= by
 simpa [b] using Real.sin_add_int_mul_two_pi (Real.pi / 2 + Real.pi) (n : $\mathbb{Z}$)
 -- Then evaluate $\sin(\pi/2 + \pi) = -\sin(\pi/2) = -1$
 simpa [Real.sin_pi_div_two] using
 (this.trans (Real.sin_add_pi (Real.pi / 2)))

-- Uniqueness of limits gives $l = -1$
have hl_eqm1 : $l = -1$:= by
 have hconst_to_l : $\text{Tendsto } (\text{fun } _ : \mathbb{N} \Rightarrow (-1 : \mathbb{R})) \text{ atTop } (\mathcal{N} l)$:= by
 simpa [hsin_b] using h2
 have hconst_to_m1 : $\text{Tendsto } (\text{fun } _ : \mathbb{N} \Rightarrow (-1 : \mathbb{R})) \text{ atTop } (\mathcal{N} (-1 : \mathbb{R}))$:=
 tendsto_const_nhds
 exact tendsto_nhds_unique hconst_to_l hconst_to_m1

-- Contradiction: $1 = -1$
have hEq : $(1 : \mathbb{R}) = -1$:= by simpa [hl_eq1] using hl_eqm1
have hNe : $(1 : \mathbb{R}) \neq -1$:= by norm_num
exact hNe hEq

```

After fixing imports, this code worked successfully! There were suggestions to change 'simp' to 'simp', but they did not improve the proof in any capacity.

For a not-so-elegant proof, Gemini 2.5 Flash generated this for the limit of  $\sin(x)$  proof:

```

``lean

import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic
import Mathlib.Topology.MetricSpace.Basic
import Mathlib.Topology.Instances.Real
import Mathlib.Data.Real.Basic
import Mathlib.Data.Int.Interval
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.NormNum
import Mathlib.Tactic.FieldSimp
import Mathlib.Algebra.Order.Floor

open Filter Topology Real Metric Set
open scoped Topology

-- Helper lemma: `sin` takes value 1 infinitely often.
lemma sin_eq_one_inf_often (M : $\mathbb{R}$) : $\exists x, x > M \wedge \sin x = 1$:= by
 -- We know $\sin(\pi/2 + 2k\pi) = 1$. We need to find k such that $\pi/2 + 2k\pi > M$.
 -- This means $2k\pi > M - \pi/2$, so $k > (M - \pi/2) / (2\pi)$.
 let k_lower_bound := (M - $\pi / 2$) / (2 * π)
 -- Choose an integer `k` strictly greater than `k_lower_bound`.
 let k_int_val : $\mathbb{Z}$:= Int.floor k_lower_bound + 1
 -- Ensure the value `k_int_val` (as a real number) is strictly greater than `k_lower_bound`.
 have h_k_int_val_gt_k_lower_bound : (k_int_val : $\mathbb{R}$) > k_lower_bound :=
 Int.lt_floor_add_one k_lower_bound

```



```

let x_val := $\pi / 2 + 2 * (k_int_val : \mathbb{R}) * \pi$
use x_val
constructor
. -- Prove $x_val > M$
calc
 x_val = $\pi / 2 + 2 * (k_int_val : \mathbb{R}) * \pi$:= rfl
 _ > $\pi / 2 + 2 * k_lower_bound * \pi$:= by
 gcongr
 norm_num -- $0 < 2$
 exact pi_pos -- $0 < \pi$
 _ = $\pi / 2 + 2 * ((M - \pi / 2) / (2 * \pi)) * \pi$:= by rw [k_lower_bound]
 _ = M := by field_simp [pi_ne_zero, two_ne_zero]
. -- Prove $\sin x_val = 1$
-- The argument to `sin` is ` $\pi / 2 + 2 * \uparrow k_int_val * \pi$ '.
-- We want to rewrite ` $2 * \uparrow k\_int\_val * \pi$ ' into ` $\uparrow (k_int_val * 2) * \pi$ '
-- for `sin_add_int_mul_two_pi`.
have h_rw_arg : $2 * (k_int_val : \mathbb{R}) * \pi = ((k_int_val * 2) : \mathbb{Z}) * \pi$:= by
 norm_cast
rw [h_rw_arg]
exact sin_add_int_mul_two_pi (k_int_val * 2) ($\pi / 2$) $\triangleright$ sin_pi_div_two

-- Helper lemma: `sin` takes value -1 infinitely often.
lemma sin_eq_neg_one_inf_often (M : $\mathbb{R}$) : $\exists x, x > M \wedge \sin x = -1$:= by
 -- We know $\sin(3\pi/2 + 2k\pi) = -1$. We need to find k such that $3\pi/2 + 2k\pi > M$.
 -- This means $2k\pi > M - 3\pi/2$, so $k > (M - 3\pi/2) / (2\pi)$.
 let k_lower_bound := $(M - 3 * \pi / 2) / (2 * \pi)$
 let k_int_val : $\mathbb{Z}$:= Int.floor k_lower_bound + 1
 have h_k_int_val_gt_k_lower_bound : $(k_int_val : \mathbb{R}) > k_lower_bound$:=
 Int.lt_floor_add_one k_lower_bound

let x_val := $3 * \pi / 2 + 2 * (k_int_val : \mathbb{R}) * \pi$
use x_val
constructor
. -- Prove $x_val > M$
calc
 x_val = $3 * \pi / 2 + 2 * (k_int_val : \mathbb{R}) * \pi$:= rfl
 _ > $3 * \pi / 2 + 2 * k_lower_bound * \pi$:= by
 gcongr
 norm_num -- $0 < 2$
 exact pi_pos -- $0 < \pi$
 _ = $3 * \pi / 2 + 2 * ((M - 3 * \pi / 2) / (2 * \pi)) * \pi$:= by rw [k_lower_bound]
 _ = M := by field_simp [pi_ne_zero, two_ne_zero]
. -- Prove $\sin x_val = -1$
have h_rw_arg : $2 * (k_int_val : \mathbb{R}) * \pi = ((k_int_val * 2) : \mathbb{Z}) * \pi$:= by
 norm_cast
rw [h_rw_arg]
exact sin_add_int_mul_two_pi (k_int_val * 2) ($3 * \pi / 2$) $\triangleright$ sin_three_pi_div_two

theorem not_tendsto_sin_at_infty (L : $\mathbb{R}$) : $\neg \text{Tendsto } \sin \text{ atTop } (\text{nhds } L)$:= by
 -- The negation of ` $\text{Tendsto } \sin \text{ atTop } (\text{nhds } L)$ ' is ` $\exists \varepsilon > 0, \forall N : \mathbb{R}, \exists x : \mathbb{R}, x > N \wedge |\sin x - L| \geq \varepsilon$ '.
 by_cases h_L_between : $L \in \text{Icc } (-1) 1$
 case pos =>

```

```

-- Case 1: $L \in [-1, 1]$. We choose $\varepsilon = 1/2$.
let $\varepsilon := (1/2 : \mathbb{R})$
-- We need to prove $\exists \varepsilon, (\varepsilon > 0 \wedge (\forall N, \dots))$.
-- So, 'use ε ' provides the ' ε '. Then 'constructor' provides the ' $\wedge$ '.
use ε
constructor
. -- Prove ' $\varepsilon > 0$ '.
 norm_num
. -- Prove ' $\forall N, \exists x, x > N \wedge |\sin x - L| \geq \varepsilon$ '.
 intro N
 -- Split L again to handle which ' $\sin x$ ' value is far from L.
 by_cases h_L_ge_zero : $L \geq 0$
 case pos =>
 -- $L \in [0, 1]$. We pick ' x ' such that ' $\sin x = -1$ '.
 obtain $\langle x, hx_N, h\sin x_neg_one \rangle := \sin_eq_neg_one_inf_often\ N$
 -- Provide the witness for ' x ' and its ' $x > N$ ' proof.
 use x, hx_N
 -- Now prove ' $|\sin x - L| \geq \varepsilon$ ' given ' $\sin x = -1$ '.
 rw [hsinx_neg_one]
 calc
 $|-1 - L| = |-(1 + L)| :=$ by ring_nf
 $_ = 1 + L \quad :=$ by rw [abs_neg]; exact abs_of_nonneg (by linarith [h_L_ge_zero])
 $_ \geq 1 + 0 \quad :=$ by gcongr
 $_ = 1 \quad :=$ by norm_num
 $_ \geq \varepsilon \quad :=$ by norm_num [ε]
 case neg =>
 -- $L \in (-1, 0)$. We pick ' x ' such that ' $\sin x = 1$ '.
 obtain $\langle x, hx_N, h\sin x_one \rangle := \sin_eq_one_inf_often\ N$
 use x, hx_N
 rw [hsinx_one]
 calc
 $|1 - L| \geq (1 - L) :=$ abs_of_pos (by linarith [h_L_ge_zero, h_L_between.1]) -- Use
h_L_between for $L \geq -1$
 $_ > 1 \quad :=$ by linarith [h_L_ge_zero]
 $_ \geq \varepsilon \quad :=$ by norm_num [ε]

case neg =>
 -- Case 2: $L \notin \text{Icc } (-1) 1$, meaning $L < -1$ or $L > 1$.
 -- We derive ' $L < -1 \vee L > 1$ ' from ' $h_L_between : L \notin \text{Icc } (-1) 1$ '.
 replace h_L_between : $L < -1 \vee L > 1 :=$ by
 rw [mem_ccc_iff] at h_L_between
 push_neg at h_L_between -- This changes ' $\neg(A \wedge B)$ ' to ' $\neg A \vee \neg B$ '
 exact h_L_between

cases h_L_between
case inl h_L_lt_neg_one =>
 -- Subcase 2a: $L < -1$. We choose $\varepsilon = (-1 - L) / 2$.
 let $\varepsilon := (-1 - L) / 2$
 -- Provide the witness for ' ε ' and its positivity proof.
 use ε
 constructor
 . -- Prove ' $\varepsilon > 0$ '.
 linarith [h_L_lt_neg_one]

```

```

. -- Prove `∀ N, ∃ x, x > N ∧ |sin x - L| ≥ ε`.
intro N
-- We need `x > N`.
obtain ⟨x, hxN⟩ := exists_gt_real N
use x, hxN
calc
 |sin x - L| = sin x - L := abs_of_nonneg (by linarith [neg_one_le_sin x, h_L_lt_neg_one])
 _ ≥ -1 - L := by linarith [neg_one_le_sin x]
 _ = (-1 - L) / 2 + (-1 - L) / 2 := by ring
 _ ≥ (-1 - L) / 2 := by linarith [half_pos.mpr (by assumption)] -- here, (by
assumption) refers to ε > 0.
case inr h_L_gt_one =>
-- Subcase 2b: L > 1. We choose ε = (L - 1) / 2.
let ε := (L - 1) / 2
-- Provide the witness for `ε` and its positivity proof.
use ε
constructor
. -- Prove `ε > 0`.
linarith [h_L_gt_one]
. -- Prove `∀ N, ∃ x, x > N ∧ |sin x - L| ≥ ε`.
intro N
obtain ⟨x, hxN⟩ := exists_gt_real N
use x, hxN
calc
 |sin x - L| = -(sin x - L) := abs_of_nonpos (by linarith [sin_le_one x, h_L_gt_one])
 _ = L - sin x := by ring
 _ ≥ L - 1 := by linarith [sin_le_one x]
 _ = (L - 1) / 2 + (L - 1) / 2 := by ring
 _ ≥ (L - 1) / 2 := by linarith [half_pos.mpr (by assumption)] -- here, (by
assumption) refers to ε > 0.

```

This one proof generated 16 errors and was, by far, one of the longest proofs generated among all the LLMs. Gemini 2.5 Flash often includes several comments to explain its reasoning, which is helpful for the user. However, when the proofs are incorrect, it raises questions about why Gemini 2.5 Flash has so many type mismatches and rewrite errors.

## 5. Discussion

Overall, LLMs are extremely useful when it comes to generating Lean4 code, but cannot be strictly relied upon. The user still needs to understand the semantics of the language to guide the LLM effectively and correct the small mistakes it tends to make. LLMs are particularly adept at establishing the overall structure of the proof, which can be time-consuming for most new and intermediate Lean4 users. However, its main weaknesses are its tendency to hallucinate theorems and tactics, its inability to debug given error messages correctly, and a reasonably lengthy human guidance process.

One of the main challenges while researching LLMs' writing Lean4 code was the limitations of an API platform like OpenRouter. Although it's beneficial to have access to a wide array of LLMs all within one website, I encountered rate limit issues with LLMs like DeepSeek Prover V2. Additionally, xAI's Grok 4 became especially hard to work with as I couldn't execute later prompts because of the rapid increase in tokens (2,000 to 10,000 to 200,000) per prompt.

So, for proofs that have an extensive error log, debugging becomes pricey.

With complex proofs, LLMs struggle with piecing together all of Mathlib's tactics and theorems. Gemini 2.5 Flash, Claude Sonnet 4, and DeepSeek Prover V2 hallucinated the most when it came to unknown identifiers and constants. Similarly, these models almost hallucinated Mathlib modules to import. So as proofs become more complex, these LLMs will continue to struggle.

One of the clear indications this research shows is that newer models, such as GPT-5, are significantly better at writing proofs in Lean4 than at writing proofs in Lean4. As new, fine-tuned models are created from already established frameworks, it's only a matter of time before proof assistants are used more frequently. What does this mean for Lean4? Gatekept by a very confusing and complex library and coding style, proof assistants open up the possibility for Lean4 to become more widespread. With how impressive GPT-5 is at writing Lean4 code, it's only a matter of time before LLMs are integrated into the proofreading process. Users can save hours by having LLMs write the repetitive and simple aspects of the proofs they're working on. In addition to keeping a significant amount of time, proof assistants can continue to be used to refer to the Mathlib library and assist beyond Lean4's current assistant, Lean Infoview.

## 6. Conclusion & Future Work

In conclusion, newer LLMs can be used as a proof assistant for writing Lean4 code. GPT-5 has demonstrated significant capabilities in writing both complex and straightforward proofs, surpassing those of other standard models. Although my assumptions about possible hallucinations are correct, having even a trim level of knowledge of Lean4 and the mathlib library makes debugging with LLMs very easy. When it comes to the choice of LLM, GPT-5 has shown promising results in proofs of varying difficulties. Although you can ultimately decide based on token cost and preference, of the ones I tested, GPT-5 is by far the most efficient at writing Lean4 code.

This research paper demonstrates how, with little knowledge of Lean4, anyone can write a simple proof with the assistance of an LLM like GPT-5. By comparing the efficiency and correctness of varying popular LLMs, GPT-5 is the go-to for both simple and complex mathematical problems. Additionally, specific prompts are not necessary for working with these LLMs, as Claude Sonnet 4 and GPT-5 were both more than capable of working with just basic instructions and a list of errors. This is significant because, before the newer releases of OpenAI and Anthropic models, LLMs were incapable of generating Lean4 code. Through my early tests in my research, GPT-4o performed extremely poorly with Lean4, and I wasn't sure if GPT-5 would show any signs of improvement. After these tests, I am much more confident in an LLM's ability to become a proof assistant or a complete proof generator in the years to come.

There are many possibilities for where this work can continue in the future. There is a more efficient way to compare these LLMs rather than having someone manually iterate through each prompting step. This research can be scaled using API calls, and there is a way to automate the provision of an error log. However, fine-tuned LLMs should become the preferred proof assistant for writing Lean4 code. DeepSeek Prover V2 is slightly outdated, but one of the most accessible of LLMs fine-tuned for writing math proofs. A better approach would be to create an LLM specifically for writing Lean4 code, through training with the Mathlib library and publicly available data on correct proofs.

## 7. LLM Use

LLMs were used to polish up the writing of this research paper.

## 8. References

OpenAI. (2025) ChatGPT [Large language model] <https://chat.openai.com/chat>

Google. (2025) Gemini [Large language model] <https://gemini.google.com/app>

OpenRouter. (2025) OpenRouter <https://openrouter.ai>

Lean Community. (2025) Mathlib [Documentation]

[https://leanprover-community.github.io/mathlib4\\_docs/index.html](https://leanprover-community.github.io/mathlib4_docs/index.html)

Lean Community. (2025) Lean4 [Compiler] <https://live.lean-lang.org/>